De-AI Prompt for Humanizing Text

https://gemini.google.com/app/f1ea3ef67915c8f7

User prompt: For both these bots, what recommended tech stacks do you recommend especially as i push them as actual github bots for my projects?

Response: Building these as actual GitHub bots requires deciding between two deployment architectures: **GitHub Actions** (where the code runs on GitHub's servers when a PR opens) or a **Hosted GitHub App** (where you host a server that listens to GitHub webhooks).

For these specific bots, building them as **GitHub Actions** is highly recommended. Bot 1 requires machine learning libraries that are annoying and expensive to host on a persistent cloud server, but they run perfectly (and for free) on GitHub's Ubuntu runners.

Here are the optimal tech stacks for both.

Bot 1: The Prose & Heuristic Scanner

Since this bot relies on statistical natural language processing (NLP), it must be built in Python. The Python ecosystem is unmatched for calculating probabilities and token distributions.

- **Language:** Python.
- **GitHub Integration:** `PyGithub`. This library wraps the GitHub REST API, allowing your script to pull the PR diff and post inline comments directly on the specific lines of documentation that fail the heuristic checks.
- **AST Parser:** `tree-sitter` alongside the `tree-sitter-typescript` grammar. It is incredibly fast, handles broken code gracefully, and has excellent Python bindings for extracting your `jsx_text` nodes.
- **Heuristics Engine:**
  - `transformers` (by Hugging Face) and `torch` (PyTorch). You will use these to temporarily load a tiny, lightweight local model (like `DistilGPT-2`) purely to run the negative log-likelihood math for the perplexity score.
  - `spaCy` or `nltk`. Use these for part-of-speech tagging and calculating the density of hedging verbs and transitional phrases.

Bot 2: The React UI Scanner

Because this bot analyzes React component structures, props, and Tailwind/utility classes, it is best built in the native JavaScript/TypeScript ecosystem. It doesn't require ML models, meaning it executes in seconds.

- **Language:** TypeScript running on Node.js.
- **GitHub Integration:** `@actions/github` (which provides an authenticated Octokit client) if building an Action. If you prefer to build a standalone GitHub App, use **Probot**—it is the official, battle-tested Node.js framework for building bots and handles all the webhook authentication automatically.
- **AST Engine:** `ts-morph` or `@typescript-eslint/parser`.
  - *Note:* While Tree-sitter is great for the Python bot, `ts-morph` wraps the native TypeScript compiler API. It is vastly superior for traversing React prop-drilling, evaluating TSX component nesting, and analyzing types.
- **Pattern Matching:** Pure TypeScript logic. You write AST visitor functions that flag structural slop (e.g., throwing a warning if a `jsx_attribute` for `className` contains more than 15 space-separated words, or if `jsx_element` nodes of type `div` are nested more than five levels deep).

User prompt: i always use python and typescript though.

Response: Then you're already set up for success—that tech split fits directly into your existing workflow.

You don't need to pick up any niche languages or extra overhead here:

- **Python for Bot 1** handles the heavier data science and NLP work (PyTorch, token perplexity calculations, and spaCy) cleanly without fighting the language.
- **TypeScript for Bot 2** gives you direct access to Node's native AST tooling (`ts-morph`) and `@actions/github` or Probot for typed, seamless React component analysis.

Which one do you want to lay the groundwork for first? We can drop straight into the Tree-sitter extraction script for the Python heuristic bot, or sketch out the `ts-morph` structural analyzer for the TypeScript React bot.

User prompt: other languages will cause issues then i take it?

Response: Not necessarily, but sticking to Python and TypeScript completely eliminates the "glue code" tax.

If you tried to build Bot 1 in Go or Rust, you'd spend half your time fighting bindings for PyTorch or pulling heavy NLP tokenizers just to calculate perplexity. Python handles that math natively. Conversely, trying to parse React JSX and TypeScript nodes in Python is painful because the official TypeScript compiler API is written in JS/TS.

Using your existing stack means:

- **Zero new tooling overhead:** You already know how to handle dependency management (`pip`/`poetry` and `npm`/`pnpm`), linters, and CI/CD runners for these languages.
- **Direct API parity:** Both ecosystems have first-class, natively maintained GitHub API wrappers (`PyGithub` and `@actions/github`).

Sticking to what you already use keeps these bots maintainable long-term instead of turning them into abandoned weekend projects.

User prompt: For these two bots, how would you consider the complexity, difficulty, and time commitment to be?

Response: Building these two bots represents two completely different types of engineering challenges. Because you are sticking to native ecosystems (Python for NLP, TypeScript for the React AST), you avoid the worst friction points, but the architectural difficulty varies heavily between them.

Here is a breakdown of the complexity, difficulty, and expected time commitment for a solid V1 of each.

Bot 1: The Heuristic Prose Scanner (Python)

This is the heavier lift of the two. You are bridging strict structural parsing (Tree-sitter) with probabilistic machine learning (transformers).

Metric	Assessment
Complexity	**High.** You must manage a two-tier architecture: extracting the nodes accurately, and then routing strings to different evaluation logic based on length (dictionary checks vs. perplexity scoring).
Difficulty	**Moderate to Hard.** The hardest part is not the code itself, but tuning the thresholds. If your perplexity threshold is too sensitive, the bot will flag every developer's natural writing as AI. If it's too loose, it catches nothing. Writing the Tree-sitter S-expression queries to isolate `jsx_text` without grabbing surrounding logic also requires significant trial and error.
Time Commitment	**25–35 Hours.** Expect to spend about 20% of your time on the GitHub Actions/PyGithub wrapper, 30% on the Tree-sitter AST extraction, and 50% purely on tuning the NLP thresholds and edge cases.

Bot 2: The React UI Scanner (TypeScript)

This project is much more straightforward. It is purely deterministic static analysis without any machine learning ambiguity.

Metric	Assessment
Complexity	**Low to Moderate.** There is no probabilistic state. A rule either passes or fails. `ts-morph` wraps the native TypeScript compiler API, meaning you get first-class types and autocomplete while you traverse the React code.
Difficulty	**Moderate.** The initial learning curve of mapping out how the TypeScript AST represents React components takes a few hours. Tracing deep prop-drilling across multiple files can be tricky, but writing the rules for messy `className` strings or deep `div` nesting is practically trivial.
Time Commitment	**10–15 Hours.** Once you set up the `@actions/github` integration and write your first AST visitor function, adding new structural checks takes minutes. The logic is entirely boolean (e.g., `if (node.getDepth() > 5) return flag;`).

The Recommended Rollout

Because Bot 2 (TypeScript) has a fixed scope and deterministic rules, it is the best place to start. Building it will force you to solve the GitHub Action deployment pipeline, handle authentication, and figure out how to post inline PR comments.

Once the CI/CD pipeline is solved on the simpler bot, you can copy that entire infrastructure over to Bot 1, leaving you free to focus entirely on the complex NLP heuristics without worrying about how to post the results back to GitHub.

User prompt: Bot 2 in particular sounds nuanced though. I specifically hate the gradients, rounded edges, and vibe coded slop website UI design, and i want to be able to have an output that is readable by an AI agent to cut it out and maybe integrate other UI skills instead. But i can easily make it too sensitive or not sensitive enough, sometimes buttons are good, sometimes the style is bad. Is there public resources and advice for this? Meanwhile in theory, bot 1 is simpler to understand on its function

Response: You are entirely right to spot the nuance here. Bot 1 is conceptually simpler because the "rules" of human prose are well-documented in NLP. Bot 2 is chasing a moving target: modern UI trends that AI models have hyper-fixated on and pushed to an extreme.

A button with rounded corners and a shadow is standard design. A `div` stacked with `bg-gradient-to-r`, `from-purple-500`, `via-pink-400`, `backdrop-blur-xl`, `shadow-2xl`, and `rounded-3xl` is the exact "vibe-coded slop" you are trying to eradicate.

If you use binary true/false rules, you will break standard UI components. To solve this, you must treat Bot 2 like an email spam filter.

The "Spam Score" Architecture

Instead of flagging a component the moment it sees a gradient, your TypeScript AST parser should assign weights to utility classes within a single DOM node.

1. **Assign Weights to AI-isms:**
    - `bg-gradient-*` (+3 points)
    - `shadow-xl` or `shadow-2xl` (+2 points)
    - `backdrop-blur-*` (+3 points)
    - `rounded-2xl` or `rounded-full` on non-interactive containers (+2 points)
    - `ring-*` combined with shadows (+2 points)
2. **Set a Threshold:** If a single HTML element accumulates a score of 5 or higher, it is mathematically confirmed as slop.

Preparing the Output for an AI Refactor

To pass this output successfully to another AI agent for a rewrite, you need a structured handoff.

When you route the flagged code into NanoGPT to hit a long-context model, you can supply the entire React file alongside a JSON payload detailing the specific AST failures. The long-context window allows the model to see the entire component structure while surgically targeting the offensive nodes.

Your payload generated by Bot 2 would look like this:

```
{
  "file": "src/components/HeroCard.tsx",
  "infraction_score": 8,
  "node_type": "div",
  "line_number": 42,
  "detected_slop": ["bg-gradient-to-r", "shadow-2xl", "backdrop-blur-md", "rounded-3xl"],
  "instruction": "Strip all detected visual classes. Refactor this node using a strict brutalist/minimalist design system. Retain only structural grid/flex utilities."
}
```

By providing the exact line numbers and offending classes, the refactoring agent doesn't have to guess what aesthetic you hate; it just acts as a text replacement tool executing the cleanup instructions.

Public Resources for Heuristic Tuning

While there is no massive open-source "Anti-AI UI" repository yet, you can reverse-engineer existing design system resources to build your scoring matrix:

1. **Reverse-Engineering "Refactoring UI":** The creators of Tailwind CSS published a book called *Refactoring UI*. AI models were heavily trained on these exact paradigms, which is why they default to them. You can use their guidelines to define the maximum acceptable limits (e.g., standard shadow depths) and penalize anything that exceeds them.
2. **ESLint Tailwind Plugins:** Look into the source code of `eslint-plugin-tailwindcss`. They have robust AST parsers for reading class strings. While their rules focus on catching conflicting classes (e.g., `p-4` `p-6`), you can fork their AST traversal logic and replace it with your point-based slop matrix.
3. **Design Token Validators:** Look into resources on "Design Token linting" (like Stylelint). Enterprise teams use these to ensure developers don't use arbitrary hex codes or massive gradients outside the company's approved design system. You are effectively building an aggressive design system enforcer.

User prompt: And would the fact that i have a raspberry pi be relevant or irrelavent?

Response: It is highly relevant, but it completely changes your deployment options. It acts as a major advantage for Bot 2, but a potential bottleneck for Bot 1.

While developing locally, the HP Victus running Manjaro has plenty of horsepower to test PyTorch inference and AST parsing. But when it comes to deploying these bots to monitor your repositories full-time, a Raspberry Pi allows you to pivot away from GitHub Actions and run them as a persistent, self-hosted service.

Here is exactly how the Pi changes the architecture for both bots.

The Advantage: A 24/7 Webhook Server (Bot 2)

If you build the React UI scanner as a GitHub App using Probot (Node.js), that bot requires a server to sit and listen for GitHub webhooks whenever a pull request is opened.

- **The standard route:** Paying for a cloud VPS (DigitalOcean, AWS) or dealing with cold-starts on serverless platforms to host the Node server.
- **The Pi route:** You run the Node.js server on the Raspberry Pi 24/7. By setting up a secure tunnel (like Cloudflare Tunnels), GitHub sends the webhook payloads directly to your desk. Node.js and `ts-morph` run perfectly on ARM architecture, making the Pi an ideal, free hosting solution for the deterministic UI scanner.

The Advantage: A Self-Hosted GitHub Runner

If you prefer the GitHub Actions route over a standalone app, you do not have to use GitHub's cloud Ubuntu servers. You can install the GitHub Actions Runner software on your Raspberry Pi and tag it as a self-hosted runner.

- When a PR opens, GitHub queues the job, and your Pi automatically pulls the code and executes the workflow.
- This gives your scripts direct access to any local network resources or custom configurations you have set up, without spending GitHub's free-tier Action minutes.

The Bottleneck: Machine Learning on ARM (Bot 1)

The Pi is phenomenal for network routing and static analysis, but it struggles with machine learning inference.

Bot 1 relies on calculating text perplexity using a local model like DistilGPT-2 via PyTorch.

- **Memory constraints:** Loading transformer models requires significant RAM. Depending on your Pi's memory configuration, it may struggle to load the model and process large documentation files simultaneously.
- **Compute constraints:** PyTorch inference on an ARM CPU is significantly slower than on an x86 processor or a dedicated cloud runner. A repository scan that takes 10 seconds on GitHub's cloud runners might take over a minute on the Pi.

If you use the Pi, the optimal hybrid approach is to host Bot 2 entirely on the Pi via Node.js, and leave Bot 1 as a standard GitHub Action running on GitHub's cloud infrastructure to handle the heavier NLP math.

User prompt: GPT sol is pushing back for RUST for speed and lightweightness. Do you agree or disagree? Specfically rust and Oxc

Response: I agree with the push for Rust in this context, and Oxc (the Oxidation Compiler) is currently the gold standard for exactly what you are trying to do. It was built specifically to solve the performance bottlenecks of parsing JavaScript and TypeScript by reimplementing the entire toolchain in Rust.

If you are building Bot 2 (the React UI scanner) to comb through entire codebases looking for specific node types, you will run into massive performance bottlenecks using JavaScript-based parsers like Babel or the native TypeScript compiler (`ts-morph`), especially when scaling to hundreds or thousands of files.

Here is why adopting Rust and Oxc makes sense for your use case, and how it compares to the TypeScript approach we discussed.

Why Oxc is the Right Tool

Oxc provides a suite of tools built on a shared, high-performance AST. For your bot, you would primarily use the `oxc_parser` and `oxc_linter` components.

1. Unmatched Speed Oxc's linter (`oxlint`) is reported to be 50 to 100 times faster than ESLint. It achieves this speed because Rust allows for memory safety without garbage collection, and Oxc allocates its AST nodes into a single memory arena for fast cleanup and cache-friendly linear memory layouts. If your bot is running in a CI/CD pipeline, an AST traversal that takes a JavaScript parser a minute might take Oxc a few hundred milliseconds.

2. Built for JavaScript and TypeScript Unlike generic AST parsers, Oxc is explicitly designed to parse ECMAScript, TypeScript, and JSX/TSX. It understands the semantics of React components natively. You do not have to write custom Tree-sitter queries to differentiate between an HTML `<div>` and a React component `<Div>`; Oxc's AST structure handles this out of the box.

3. It Has a Linter Architecture You Can Hijack You don't just want to parse the code; you want to flag it. Oxc already has a production-ready linter (`oxlint`) that uses this AST. Instead of building an AST traversal engine from scratch, you can write custom Oxc linter rules in Rust to look for your specific UI "slop."

The Trade-off: Leaving the JS Ecosystem

The major drawback of switching from `ts-morph` to Oxc is the language ecosystem switch.

- **TypeScript (`ts-morph`)**: You are writing your React analyzer in the same language as the codebase it analyzes. The tooling is familiar, and Node runs everywhere.
- **Rust (Oxc)**: You must write your traversal logic in Rust. While Oxc provides Rust crates specifically for traversal (using visitor patterns), you have to manage Rust toolchains, deal with the borrow checker, and compile binaries for your deployment targets (like your Raspberry Pi).

The Verdict

If you want the absolute fastest, most resource-efficient tool to rip through a massive React codebase and flag nested `div`s and excessive `className` strings, **Oxc is the correct architectural choice**.

If you decide to go the Rust route for Bot 2, you are essentially building a custom `oxlint` plugin. You would define a visitor struct in Rust that traverses the Oxc AST, looks for `JSXAttribute` nodes named `className`, calculates your "slop score," and emits diagnostic warnings.

Sources

1. Oxc - Rust Utilities
2. OXC JavaScript Tooling: 100x Faster Rust-Based Development Tools - Ali Soueidan
3. AST | The JavaScript Oxidation Compiler - Oxc

User prompt: I am unfamiliar with rust, is there its own equivalent of valgrind and other tools i need to think about?

Response: Because you're asking about tools like Valgrind in the context of Rust, the good news is that Rust was specifically designed to make these tools mostly unnecessary. Rust's strict compiler, ownership rules, and borrow checker guarantee at compile time that entire classes of memory errors—like use-after-free, double-free, and data races—cannot happen.

However, if you ever step outside of "safe" Rust and use the `unsafe` keyword (which allows raw pointer manipulation and calling C/C++ libraries), those compile-time guarantees are bypassed. In those cases, you do need tools.

Here is the tool ecosystem in Rust compared to Valgrind:

1. Miri (The Rust Standard)

If you write `unsafe` Rust, **Miri** is the tool you use. Miri is an official, built-in interpreter for Rust's Mid-level Intermediate Representation (MIR).

- **How it works:** Instead of running the compiled binary directly on your CPU (like Valgrind does), Miri executes your Rust code inside its own virtual machine.
- **What it catches:** Miri is incredibly strict. It catches out-of-bounds memory accesses, use-after-free errors, memory leaks, invalid uninitialized data, and violations of Rust's specific aliasing rules (like having two mutable references to the same data).
- **Advantage over Valgrind:** Valgrind is great for C/C++, but it doesn't understand Rust's specific rules (Undefined Behavior). Miri is built specifically to enforce the Rust specification.
- **How to use it:** It's baked into the Cargo ecosystem. You just run `cargo miri test` instead of `cargo test`.

2. LLVM Sanitizers (AddressSanitizer, ThreadSanitizer, etc.)

Because the Rust compiler (`rustc`) uses LLVM as its backend (just like Clang for C++), you get access to all the standard LLVM sanitizers.

- **How they work:** Unlike Miri (which runs an interpreter), sanitizers actually compile your code into a binary but inject runtime checks directly into the machine code.
- **The Options:** You can use AddressSanitizer (for memory bounds), LeakSanitizer (for memory leaks), ThreadSanitizer (for data races), and MemorySanitizer (for uninitialized memory).
- **Advantage over Valgrind:** Sanitizers are much faster than Valgrind because they are compiled into the binary, rather than emulating the CPU at runtime.
- **How to use them:** They currently require the Rust Nightly toolchain. You compile with a flag like `RUSTFLAGS="-Zsanitizer=address" cargo run`.

Do You Need Valgrind?

Technically, you *can* run a compiled Rust binary through Valgrind, but almost no one does in the modern Rust ecosystem. Miri is more precise for catching Rust-specific undefined behavior, and the LLVM sanitizers are significantly faster and catch more edge cases if you are interacting with C code.

For building an AST parser like we discussed with Oxc, you won't need to worry about any of these tools unless you start writing heavy `unsafe` blocks. Pure, safe Rust handles the memory management for you.

Sources

1. Mitigating False Positives in Static Memory Safety Analysis of Rust Programs via Reinforcement Learning - arXiv
2. Making Unsafe Rust a Little Safer: Tools for Verifying Unsafe Code, Including Libraries in C and C++ - Colin Breck
3. rust-lang/miri: An interpreter for Rust's mid-level intermediate representation - GitHub
4. What's "new" in Miri (and also, there's a Miri paper!) - ralfj.de
5. sanitizer - The Rust Unstable Book
6. Building better software with better tools: sanitizers versus valgrind - Daniel Lemire's blog

---

User prompt: As i am working on bot 2, im brainstorming bot 1, are you sure there arent any pretrained models and if its even worth training a new model for ai flagging detection? And instead should just make a wrapper for UI and documentation text instead?

Response: Yes, there are absolutely pre-trained open-source models explicitly fine-tuned for AI text detection, and you should definitely use them instead of training your own from scratch.

Training a new classification model is a massive time sink. The open-source community has already done the heavy lifting of gathering hundreds of thousands of human vs. AI text samples and fine-tuning robust models on top of architectures like RoBERTa and DeBERTa.

The Best Pre-Trained Models for Bot 1

If you are building the heuristic prose scanner (Bot 1), you don't need to manually calculate perplexity and write threshold logic. You can just load one of these pre-trained classification models via Hugging Face.

1. TMR (Target Mining RoBERTa) by Oxidane This is currently one of the most sophisticated open-source detectors available.

- **Architecture:** `RoBERTa-base` (125M parameters).
- **Why it's good:** It was trained specifically using a "Self-Hard-Negative" strategy on the RAID dataset (a major benchmark for AI detection). This means the researchers actively trained it on text that is *difficult* to classify, driving its false positive rate down to around 2.6%.
- **Resource Cost:** Very lightweight. It runs easily on standard CPUs and won't choke your GitHub Actions runner.

2. AI Detector by SuperAnnotate

- **Architecture:** `RoBERTa Large`.
- **Why it's good:** This model was explicitly trained to catch text generated by major modern LLM families (GPT, LLaMA, Anthropic, and Mistral). They trained it on a mix of Wikipedia, Reddit ELI5, and scientific papers, making it highly effective at spotting AI slop in technical documentation and README files.
- **Resource Cost:** Because it is the "Large" variant, it requires more memory to load, but it is still highly manageable for a CI pipeline.

3. Base RoBERTa / DeBERTa Implementations If you want to build a custom classifier later, both `RoBERTa` and `DeBERTa` are available natively via the `transformers` library. DeBERTa, in particular, separates a word's content from its position during processing, making it incredibly good at understanding the actual semantic structure of a sentence rather than just string-matching.

How This Changes Your Bot 1 Architecture

Using a pre-trained model makes Bot 1 drastically simpler. You no longer need to calculate burstiness or N-gram frequencies manually.

Instead, the pipeline looks like this:

1. **Extract:** Your Python script uses Tree-sitter to pull the `jsx_text` or docstrings.
2. **Classify:** You pass that string directly into the Hugging Face `pipeline` using one of the pre-trained models above.
3. **Result:** The model returns a simple probability score (e.g., `{"label": "AI-Generated", "score": 0.98}`). If the score is high enough, your bot flags it.

Should you just use a wrapper instead?

If you want to skip hosting models entirely, you *could* write a wrapper around a commercial API (like GPTZero or Originality.ai). However, for a GitHub Bot scanning entire codebases:

- **Cost:** Commercial APIs charge per word or token. Scanning a massive repository of documentation on every PR will get expensive fast.
- **Privacy:** Sending your proprietary codebase documentation to a third-party detection service on every commit is generally a bad idea for security.

Running a pre-trained RoBERTa model locally inside your GitHub Action (or on your Manjaro laptop/Raspberry Pi) is free, secure, and requires only a few lines of Python using the `transformers` library.

Sources

1. Oxidane/tmr-ai-text-detector - Hugging Face
2. SuperAnnotate/ai-detector - Hugging Face
3. transformers/docs/source/en/model_doc/deberta.md at main - GitHub

---

User prompt: So i can run one of these models on my pi, build a wrapper for my pi and release an app only i can run because it specifically targets my pi ssh? (I am perfectly fine with this!)

Response: Yes, absolutely! You can 100% build this, and running it as your own personal self-hosted service on the Raspberry Pi is actually a classic, super clean dev setup.

Since you only care about using it for your own projects, you don't have to worry about multi-tenant security, API rate limits, or paying for cloud hosting.

Here is how you can set up a lightweight, hyper-private pipeline for your Pi.

1. Optimize the Model for the Pi (ONNX Runtime)

Running heavy PyTorch models directly on ARM CPUs can be sluggish if you load the whole framework into RAM every time.

To make RoBERTa or TMR run lightning-fast on the Pi:

- Export the PyTorch model to **ONNX (Open Neural Network Exchange)** format.
- Quantize the model weights down to **INT8** (8-bit integers).

This reduces a 125M parameter RoBERTa model from ~500MB down to **~120MB**, and drops CPU RAM usage to almost nothing. Using `onnxruntime` on the Pi, classification latency drops to **under 50ms per string**.

2. Connecting to Your Pi: SSH vs. Tailscale API

You have two main choices for how your local apps or scripts talk to the Pi:

Approach A: Raw SSH Tunnel / Command Execution

Your local script (or GitHub Action runner) triggers a command over SSH directly:

```
ssh pi@your-pi-ip "python3 /home/pi/scanner/detect.py --text 'Never committed, never published'"
```

- **Pros:** Extremely simple to test from a terminal.
- **Cons:** Opening an SSH connection on every single string or PR creates unnecessary overhead.

Approach B: Tailscale + FastAPI (Recommended)

Instead of raw SSH, run a tiny **FastAPI** web server on the Pi that keeps the ONNX model loaded in memory 24/7. Put your Pi on a private **Tailscale** mesh network.

`[Local Laptop / CI] ──(Private Tailscale Mesh)──> [Raspberry Pi: FastAPI Endpoint]`

- **Zero Public Exposure:** Tailscale encrypts the connection peer-to-peer. You don't have to port-forward SSH or expose ports to the public internet.
- **Instant Inferences:** The model stays warm in Pi memory. Your local scripts just make a standard HTTP POST request to `[http://100.](http://100.)x.y.z:8000/detect`.

3. The Local Wrapper App

On your laptop or in your git hooks, you write a lightweight wrapper CLI (or a custom extension) that:

1. Grabs the text from your local repository or PR diff.
2. Sends the batch of extracted strings over your private network to the Pi endpoint.
3. Receives the JSON response containing the confidence scores.
4. Outputs the results directly in your terminal or editor interface.

This gives you a completely free, private, self-hosted AI detection microservice that runs entirely on hardware sitting on your desk.

User prompt: Lastly im thinking about a bot 3 which is the simpliest of all which is a check on if certain popular vibe coded files are properly gitignored or not, just a basic check. .opencode, claude.md, .claude, .codex., .agents, the ADR folder from /grill-me, etc

Response: Bot 3 is a fantastic idea and is incredibly easy to build. The problem with modern AI coding agents (Claude Code, OpenCode, Cursor, Codex, etc.) is that they drop hidden configuration files and local memory into your project root.

If these files are not ignored, they pollute your git history, leak your personal API configurations, or force your entire team to download a 300-line `CLAUDE.md` context file that was only meant for your machine.

Here is a simple bash script you can run locally or wire into a Git pre-commit hook.

The AI-Ignore Checker Script

This script scans your project for the most common agentic configuration files and checks if they are properly listed in your `.gitignore`.

```
#!/bin/bash
# check_ai_ignores.sh
# A script to ensure AI agent config files are properly gitignored

# List of common AI agent files and directories to check
AI_FILES=(
  ".claude"
  "CLAUDE.md"
  "CLAUDE.local.md"
  ".agents"
  "AGENTS.md"
  ".opencode"
  "opencode.json"
  ".cursor"
  ".cursorrules"
  ".codex"
  ".mcp.json"
  "claude.json"
  "grill-me-ADR" # Or whatever folder you use for /grill-me output
)

# Colors for output
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[0;33m'
NC='\033[0m' # No Color

echo "Scanning for exposed AI configuration files..."
FAILED=0

# Check if .gitignore exists
if [ ! -f .gitignore ]; then
    echo -e "${YELLOW}Warning: No .gitignore file found in this directory.${NC}"
    exit 0
fi

for file in "${AI_FILES[@]}"; do
    # Only check if the file/folder actually exists in the project
    if [ -e "$file" ]; then
        # Check if git tracks the file (meaning it is NOT ignored)
        # git check-ignore returns 0 if ignored, 1 if not ignored
        if ! git check-ignore -q "$file"; then
            echo -e "${RED}FAILURE: '$file' exists but is NOT in .gitignore!${NC}"
            FAILED=1
        else
            echo -e "${GREEN}OK: '$file' is properly ignored.${NC}"
        fi
    fi
done

if [ $FAILED -eq 1 ]; then
    echo -e "\n${RED}Error: Exposed AI files detected. Please add them to your .gitignore before committing.${NC}"
    exit 1
else
    echo -e "\n${GREEN}Success: No exposed AI config files found.${NC}"
    exit 0
fi
```

How to use it

1. Save it as `check_ai_ignores.sh` in your project root.
2. Run `chmod +x check_ai_ignores.sh` to make it executable.
3. Run `./check_ai_ignores.sh`.

The Pre-Commit Hook (Automating it)

You can force Git to run this check automatically every time you try to commit. Copy the script into your `.git/hooks/pre-commit` file and ensure it is executable. If the script finds a `CLAUDE.md` or `.opencode` directory that isn't ignored, it will block the commit entirely, preventing the slop from ever hitting your repository.

User prompt: is it worth making bots 1-3 seperately and then making a superskill that installs all onto github and my code repos?

Response: Yes, it is entirely worth it. Instead of reinventing the wheel to install your tools across multiple repositories, you can leverage a "superskill" installer workflow. This is a common and highly effective pattern in modern AI agent architectures.

By building Bots 1, 2, and 3 as modular components and then tying them together with a composite GitHub Action or an Agent Skill installer, you create a single, portable enforcement layer that instantly protects any codebase you attach it to.

Here is exactly how you design the superskill architecture to deploy all three bots automatically.

The Architecture: "The Refactor Toolkit"

You will build a single, centralized repository (e.g., `Vincent-Refactor-Toolkit`) that houses the deployment logic for your bots. This repo will export two things:

1. **A GitHub Composite Action:** To enforce the rules in your CI/CD pipelines.
2. **An Agent Skill (`SKILL.md`):** To inject these rules directly into your local coding agents (like OpenCode or Claude Code) so they don't generate the slop in the first place.

Part 1: The GitHub Composite Action Installer

GitHub allows you to create "Composite Actions". Unlike standard workflows, a composite action lets you bundle multiple scripts, environment setups, and node modules into a single step that you can call from *any* other repository.

In your central toolkit repo, create an `action.yml` file:

```
# action.yml (in Vincent-Refactor-Toolkit)
name: 'Vincent AI Refactor Suite'
description: 'Runs AI heuristic prose scanning, React UI slop detection, and gitignore enforcement.'

runs:
  using: "composite"
  steps:
    - name: Run Bot 3 (Gitignore Check)
      shell: bash
      run: |
        curl -s https://raw.githubusercontent.com/VincentHuang/Refactor-Toolkit/main/bots/bot3-ignores.sh | bash

    - name: Setup Node for Bot 2 (React UI Scanner)
      uses: actions/setup-node@v4
      with:
        node-version: '20'

    - name: Run Bot 2 (ts-morph slop detector)
      shell: bash
      run: npx ts-node https://raw.githubusercontent.com/VincentHuang/Refactor-Toolkit/main/bots/bot2-react.ts

    - name: Run Bot 1 (Heuristic Prose Scanner via Pi)
      shell: bash
      # This curls your Tailscale Pi endpoint with the extracted docs
      run: |
        curl -s https://raw.githubusercontent.com/VincentHuang/Refactor-Toolkit/main/bots/bot1-extract.sh | bash
```

How you use it: Now, in any of your personal projects, you just add four lines to your standard `.github/workflows/pr.yml` file. You don't have to copy-paste the bots into every repo.

```
jobs:
  clean-code:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: VincentHuang/Refactor-Toolkit@main # Installs and runs all 3 bots instantly
```

Part 2: The Local Agent Superskill (`SKILL.md`)

GitHub Actions catch the slop *after* it's committed. To stop your local AI agents from writing it in the first place, you package these rules as an Agent Skill.

The Agent Skill standard (using `SKILL.md`) is a portable format that injects system instructions and workflow commands directly into local tools like OpenCode.

Create a `SKILL.md` in your central toolkit repo:

```
---
name: vincent-refactor-rules
description: Enforces strict UI, prose, and configuration rules across all code generation.
---

# Vincent's Refactor Rules

You are acting within a strictly enforced environment. Before generating code or text, you MUST obey the following constraints:

## 1. UI Constraints (Bot 2 Pre-Emption)
- NEVER use stacked gradients (`bg-gradient-*`).
- NEVER use excessive shadows or rounded corners (`rounded-3xl`, `shadow-2xl`) on standard containers.
- Write minimal, brutalist UI structures. Avoid "vibe-coded" Tailwind slop.

## 2. Prose Constraints (Bot 1 Pre-Emption)
- NEVER use words like: delve, tapestry, bustling, testament, crucial.
- NEVER use em-dashes (—) in UI copy.
- Write short, asymmetrical sentences.
```

```
## 3. Config Constraints (Bot 3 Pre-Emption)
- Before creating any `.claude`, `.opencode`, or `CLAUDE.md` files, you MUST verify they are listed in the `.gitignore`. Do not proceed until they are ignored.
```

If you are using a tool like the `skills` CLI, you can install this directly into your local agent environment with a single command (e.g., `npx skills add VincentHuang/Refactor-Toolkit`).

The Result

By building them separately but deploying them centrally, you get the best of both worlds:

1. **Agent Skills** stop the AI from generating slop while you type.
2. **The Composite Action** catches anything that slips through when you push to GitHub.

Sources

1. GitHub - iliaal/ai-skills: Curated collection of agent skills for AI coding assistants.
2. Reusing workflow configurations - GitHub Docs
3. Understanding and Using Composite Actions in GitHub - Earthly Blog

---